Listing 9.7 shows the pseudocode for the `findMin` method: the first difference you can see with respect to *BST*'s version, is that we need to pass the index of the coordinate for which we look for the minimum (for instance, we could call `findMin(root, 2)` to search the node in the whole tree with the minimum value for the 3rd coordinate – assuming `k>=3`).

This greater complexity in the `findMin` method, unfortunately reflects on its running time: it can't be logarithmic like for *BST*s, because we are traversing all branches for all levels but the ones matching `coordinateIndex`, so in `(k-1)/k` cases.

And, in fact, it turns out that the running time for `findMin` is `O(n(k-1)/k)`: if `k==2`, this means `O(n1/2) = O()`, which is not as good as logarithmic, but still sensibly better than a full scan; as `k` grows, though, this value gets closer and closer to `O(n)`.

The enhanced version of `findMin` above solves the issue with the coordinates; now you might hope that plugging it in the regular *BST*'s `remove` is enough, but that's unfortunately not the case: there is another issue that complicates things a bit further.

If you go back to figure 9.15B, for *BST*s there were two lucky cases for which deleting a node was easy: deleting a leaf, but also deleting a node with only one child.

For k-d trees, only leaves can be deleted easily: unfortunately, even if the node `N` to be deleted has only one child `C`, we can't just replace `N` with its child, because this